

Objetivos:

- I. Criação do projeto e instalação das dependências.
- II. Estrutura do projeto.
- III. Código completo em TypeScript.
- IV. Desenvolvendo o FrontEnd.
- V. Executando o projeto.

I. Criação do projeto e instalação das dependências

A princípio, iremos criar uma pasta denominada **cadastro-clientes**. Logo em seguida, aplicaremos o comando:

```
npm init -y
```

Iremos instalar as dependências:

```
npm install express mongoose  
npm install -D typescript ts-node-dev @types/node @types/express
```

Explicação das dependências:

- express → framework para criar servidor HTTP e rotas da API.
- mongoose → ODM (Object Data Modeling) que facilita trabalhar com MongoDB usando modelos/classes.
- typescript → linguagem tipada para o Node.js.
- ts-node-dev → roda arquivos TypeScript no Node.js em modo desenvolvimento (reinicia automático).
- @types/node → definições de tipos do Node.js (para o TypeScript entender os módulos nativos).
- @types/express → definições de tipos do Express.

Configurar o TypeScript:

```
npx tsc --init
```

No tsconfig.json, alterar:

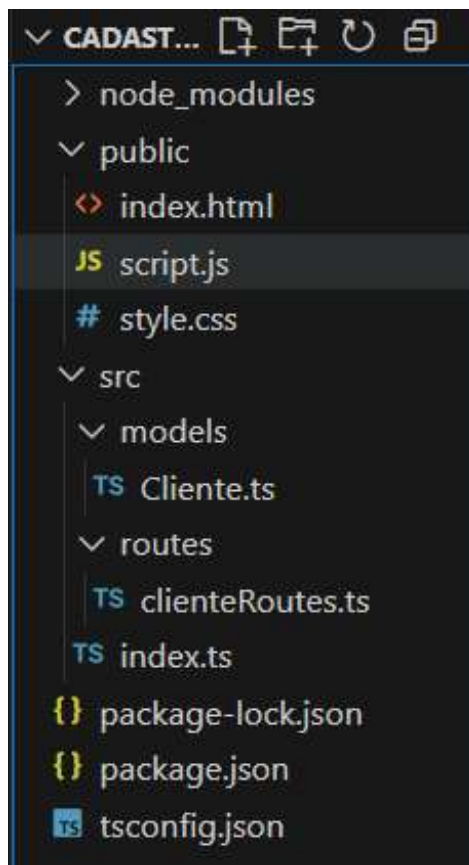
```
"target": "ES2020",  
"module": "CommonJS",  
"rootDir": "./src",  
"outDir": "./dist",  
"strict": true,  
"esModuleInterop": true,  
"moduleResolution": "Node",  
"verbatimModuleSyntax": false
```

No package.json, inserir no script:

```
"scripts": {  
  "test": "echo \"Error: no test specified\" && exit 1",  
  "dev": "ts-node-dev src/index.ts"  
},
```

II. Estrutura do projeto

Nosso projeto terá a seguinte estrutura:



Cada pasta tem a sua função dentro do projeto.

`node_modules/`

- Criada automaticamente pelo npm install.
- Guarda todas as dependências do projeto (express, mongoose, typescript, etc).
- Nunca editar manualmente

`public/`

- Onde fica o front-end básico (HTML, CSS e JS).
- O Express serve esses arquivos para o navegador (`app.use(express.static(...))`).
- Você acessa pelo navegador em <http://localhost:3000/>.

`src/models/`

- Contém os modelos (schemas) do Mongoose.
- Cada modelo corresponde a uma coleção no MongoDB.
- Exemplo: `Cliente.ts` define a estrutura do documento `Cliente` (nome, email, telefone, etc).
- Ao salvar um cliente → vira um documento dentro da coleção `clientes` no MongoDB Compass.

`src/routes/`

Onde ficam as rotas da API (endpoints HTTP).

- Exemplo: `clienteRoutes.ts` tem:
- `POST /clientes` → cadastrar cliente.
- `GET /clientes` → listar clientes.
- Separar as rotas deixa o código mais organizado e escalável.

`src/index.ts`

Arquivo principal do servidor.

Funções:

- Configura o Express.
- Conecta ao MongoDB (via Mongoose).
- Registra as rotas (`app.use("/clientes", clienteRoutes)`).
- Inicia o servidor (`app.listen(3000)`).

`dist/`

- Pasta de saída do código compilado em JavaScript.
- É gerada quando você roda `npx tsc`.
- O Node não executa TypeScript diretamente, então o TypeScript converte para `.js` aqui.

`package.json`

Arquivo de configuração do Node.js.

Guarda:

- Nome, versão e autor do projeto.
- Scripts (`"dev": "ts-node-dev src/index.ts"`).
- Dependências (express, mongoose, etc).

`tsconfig.json`

Configura o compilador TypeScript.

Define coisas como:

- Versão do JavaScript de saída (target: "ES2020").
- Pasta de entrada (rootDir: "./src") e saída (outDir: "./dist").
- Compatibilidade com módulos (module: "CommonJS" ou ESNext).

III. Código completo em TypeScript

Vamos criar o modelo Cliente `src/models/Cliente.ts`:

```
import mongoose, { Schema, Document } from "mongoose";

// Interface que define como será a estrutura de um Cliente no TypeScript
export interface ICliente extends Document {
  nome: string;
  email: string;
  telefone: string;
  criadoEm: Date;
}

// Definindo o esquema (estrutura do documento no MongoDB)
const ClienteSchema: Schema = new Schema({
  nome: { type: String, required: true }, // campo obrigatório
  email: { type: String, required: true, unique: true }, // não pode repetir
  telefone: { type: String, required: true },
  criadoEm: { type: Date, default: Date.now }, // data automática
});

// Exportando o modelo para ser usado no CRUD
export default mongoose.model<ICliente>("Cliente", ClienteSchema);
```

Criar as rotas de Cliente `src/routes/clienteRoutes.ts`:

```
import { Router, Request, Response } from "express";
import Cliente from "../models/Cliente";

const router = Router();

// Rota POST -> Criar novo cliente

router.post("/", async (req: Request, res: Response) => {
  try {
    const novoCliente = new Cliente(req.body);
    const clienteSalvo = await novoCliente.save(); // salva no MongoDB
    res.status(201).json(clienteSalvo);
  } catch (erro: unknown) {
    // ✅ Tratamento seguro do erro (unknown)
    if (erro instanceof Error) {
      res.status(400).json({ erro: erro.message });
    } else {
      res.status(400).json({ erro: String(erro) });
    }
  }
});
```

```
// Rota GET -> Listar todos os clientes cadastrados

router.get("/", async (_req: Request, res: Response) => {
  try {
    const clientes = await Cliente.find(); // busca todos os clientes
    res.json(clientes);
  } catch (erro: unknown) {
    if (erro instanceof Error) {
      res.status(500).json({ erro: erro.message });
    } else {
      res.status(500).json({ erro: String(erro) });
    }
  }
});

export default router;
```

Criar o servidor principal `src/index.ts`:

```
import express from "express";
import mongoose from "mongoose";
import path from "path";
import clienteRoutes from "../routes/clienteRoutes";

const app = express();

// Middleware para JSON
app.use(express.json());

// Conexão MongoDB
mongoose.connect("mongodb://127.0.0.1:27017/cadastroDB")
  .then(() => console.log("✅ Conectado ao MongoDB Compass"))
  .catch((erro) => console.error("❌ Erro ao conectar:", erro));

// Rotas API
app.use("/clientes", clienteRoutes);

// Servir front-end da pasta public
app.use(express.static(path.join(__dirname, "../public")));

// Inicia servidor
app.listen(3000, () => {
  console.log("🚀 Servidor rodando em http://localhost:3000");
});
```


IV. Desenvolvendo o FrontEnd

Criando o HTML através do arquivo `public/index.html`

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Cadastro de Clientes</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Cadastro de Clientes</h1>

  <!-- Formulário de cadastro -->
  <form id="form-cliente">
    <input type="text" id="nome" placeholder="Nome" required>
    <input type="email" id="email" placeholder="E-mail" required>
    <input type="text" id="telefone" placeholder="Telefone" required>
    <button type="submit">Cadastrar</button>
  </form>

  <h2>Lista de Clientes</h2>
  <ul id="lista-clientes"></ul>

  <script src="script.js"></script>
</body>
</html>
```

Criando o estilo da nossa página através do arquivo `public/style.css`

```
body {
  font-family: Arial, sans-serif;
  margin: 20px;
}

h1, h2 {
  text-align: center;
}

form {
  display: flex;
  gap: 10px;
  justify-content: center;
  margin-bottom: 20px;
}

input, button {
  padding: 8px;
}

ul {
  list-style: none;
  padding: 0;
}

li {
  padding: 6px;
  border-bottom: 1px solid #ddd;
}
```

Criando o script da página através do arquivo `public/script.js`

```
const API_URL = "http://localhost:3000/clientes";
const form = document.getElementById("form-cliente");
const lista = document.getElementById("lista-clientes");

// Função para carregar clientes
async function carregarClientes() {
  const resp = await fetch(API_URL);
  const clientes = await resp.json();

  lista.innerHTML = ""; // limpa lista
  clientes.forEach(cliente => {
    const li = document.createElement("li");
    li.textContent = `${cliente.nome} - ${cliente.email} - ${cliente.telefone}`;
    lista.appendChild(li);
  });
}
```

```
// Função para cadastrar cliente
form.addEventListener("submit", async (e) => {
  e.preventDefault();

  const novoCliente = {
    nome: document.getElementById("nome").value,
    email: document.getElementById("email").value,
    telefone: document.getElementById("telefone").value
  };

  await fetch(API_URL, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(novoCliente)
  });

  form.reset();
  carregarClientes();
});

// Carrega clientes ao abrir página
carregarClientes();
```

V. Executando o projeto

Para executar o projeto iremos utilizar o comando

```
npm run dev
```

```
> cadastro-clientes@1.0.0 dev
```

```
> ts-node-dev src/index.ts
```

```
[INFO] 20:33:48 ts-node-dev ver. 2.0.0 (using ts-node ver. 10.9.2, typescript ver. 5.9.2)
```

```
🚀 Servidor rodando em http://localhost:3000
```

```
✅ Conectado ao MongoDB Compass
```